Assignment-8.1

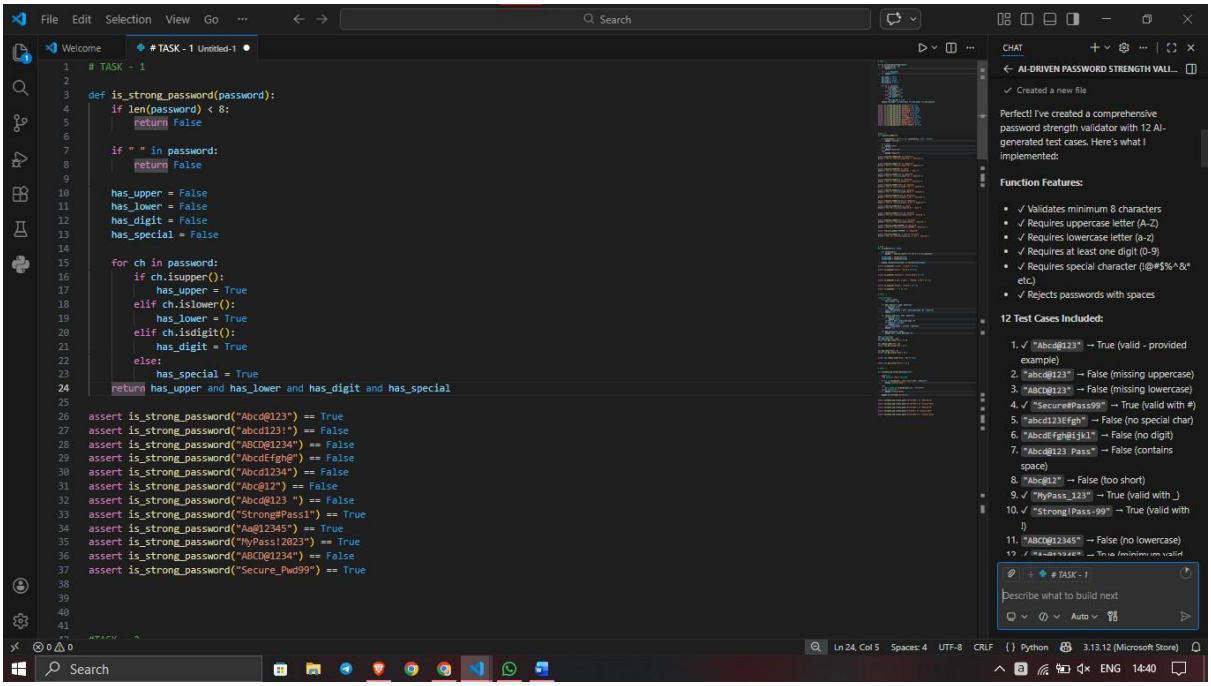
R Nikhil Kumar

2303A52260

Batch-44

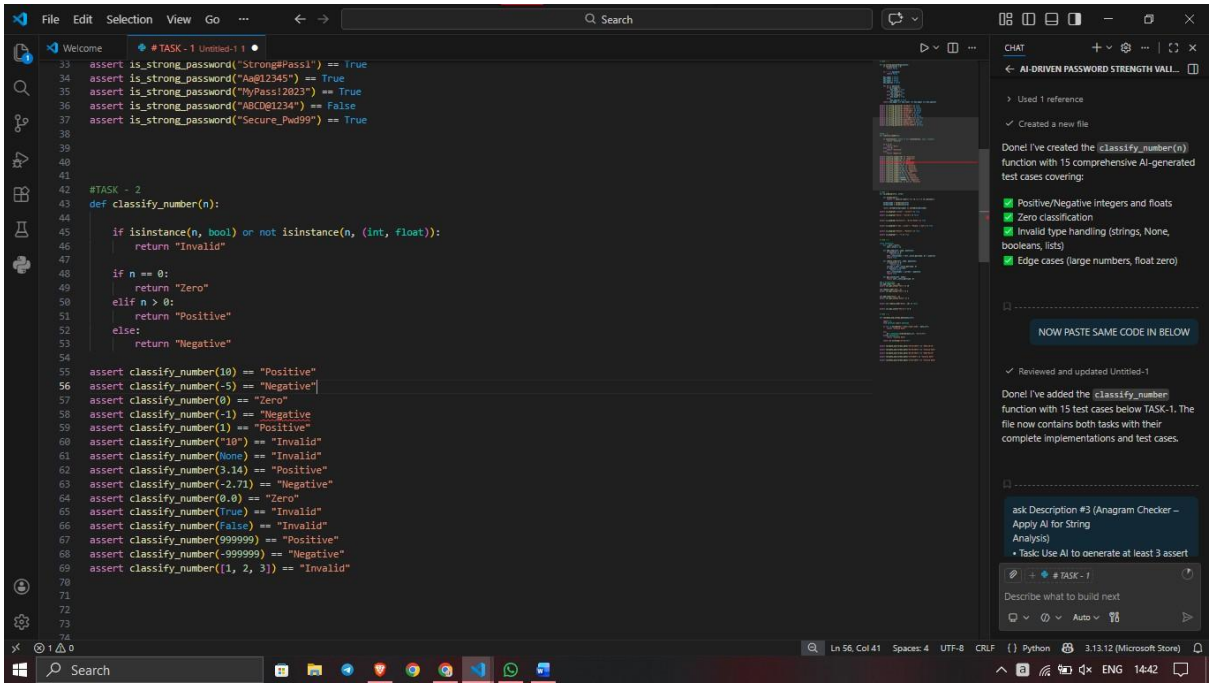
Task 1: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.

Code:



Task 2: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.

Code:



Task 3: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.

```python
# TASK - 3
def is_anagram(str1, str2):

    def normalize(s):
        return "".join(ch.lower() for ch in s if ch.isalnum())

    normalized1 = normalize(str1)
    normalized2 = normalize(str2)

    return sorted(normalized1) == sorted(normalized2)

assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
assert is_anagram("A man, a plan!", "Panama: a man") == True
assert is_anagram("Python", "Python") == True
assert is_anagram("", "") == True
```

Task Description #3 (Anagram Checker – Apply AI for String
Analysis)
• Task: Use AI to generate at least 3 assert test cases for
is_anagram(str1, str2) and implement the function.
• Requirements:
o Ignore case, spaces, and punctuation.
o Handle edge cases (empty strings, identical words).
Example Assert Test Cases:
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
Expected Output #3:
• Function correctly identifying anagrams and passing all AI-
generated tests.

Task 4: Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Code:

```python
# TASK - 4

class Inventory:
    def __init__(self):
        self._stock = {}

    def add_item(self, name, quantity):
        if quantity <= 0:
            return False
        self._stock[name] = self._stock.get(name, 0) + quantity
        return True

    def remove_item(self, name, quantity):
        if quantity <= 0:
            return False
        current = self._stock.get(name, 0)
        if quantity > current:
            return False
        self._stock[name] = current - quantity
        return True

    def get_stock(self, name):
        return self._stock.get(name, 0)

inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
assert inv.remove_item("Book", 10) == False
assert inv.get_stock("Pencil") == 0
```

Restore Checkpoint

Task Description #4 (Inventory Class – Apply AI to Simulate Real-
World Inventory System)
• Task: Ask AI to generate at least 3 assert-based tests for an
Inventory class with stock management.
• Methods:
o add_item(name, quantity)
o remove_item(name, quantity)
o get_stock(name)
Example Assert Test Cases:
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
Expected Output #4:
• Fully functional class passing all assertions.
Task Description #5 (Date Validation & Formatting – Apply AI for
Data Validation)
• Task: Use AI to generate at least 3 assert test cases for
validate_and_format_date(date_str) to check and convert
dates.
• Requirements:

Task 5: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

Code:

```python
# TASK - 5

def validate_and_format_date(date_str):

    import re
    from datetime import datetime

    if not re.fullmatch(r"\d{2}/\d{2}/\d{4}", date_str):
        return "Invalid Date"

    try:
        dt = datetime.strptime(date_str, "%m/%d/%Y")
    except ValueError:
        return "Invalid Date"

    return dt.strftime("%Y-%m-%d")


assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("02/29/2024") == "2024-02-29"
assert validate_and_format_date("2/9/2026") == "Invalid Date"
assert validate_and_format_date("13/01/2024") == "Invalid Date"
```

Expected Output #4:
• Fully functional class passing all assertions.
Task Description #5 (Date Validation & Formatting – Apply AI for
Data Validation)
• Task: Use AI to generate at least 3 assert test cases for
validate_and_format_date(date_str) to check and convert
dates.
• Requirements:
o Validate "MM/DD/YYYY" format.
o Handle invalid dates.
o Convert valid dates to "YYYY-MM-DD".
Example Assert Test Cases:
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"
Expected Output #5:
• Function passes all AI-generated assertions and handles edge
cases